

Technical Report: AI-Driven Network Intrusion Detection System

1. Problem Overview

The proliferation of sophisticated cyber threats has rendered traditional, purely signature-based Intrusion Detection Systems (IDS) insufficient. While signature-based systems are highly effective at identifying known threats with minimal false positives, they are inherently reactive, requiring prior knowledge of an attack payload. This leaves networks vulnerable to zero-day exploits and polymorphic malware that dynamically alter their signatures to evade detection. To bridge this critical security gap, modern network defense requires a shift toward behavioral analysis.

The primary objective of this project is to design, develop, and evaluate a working Machine Learning (ML) anomaly-detection prototype capable of ingesting raw network flow data, extracting meaningful statistical features, and accurately classifying traffic as benign or malicious. This ML approach is designed to eventually function as the behavioral layer in a **hybrid** detection architecture, combining the speed of signature matching with the adaptability of machine learning.

2. Objectives

- **Architecture Design:** Develop a lightweight, flow-based IDS that evaluates network traffic without relying on Deep Packet Inspection (DPI), thereby preserving user privacy and reducing computational overhead.
- **Feature Engineering:** Extract and utilize statistical flow features (e.g., duration, byte rate, packet length) to mathematically differentiate normal user behavior from malicious network activity.
- **Model Implementation:** Train a robust Machine Learning classifier to identify volumetric anomalies and subtle, low-and-slow reconnaissance techniques.
- **Real-time Simulation:** Build a Command Line Interface (CLI) application capable of ingesting network flows and providing real-time, explainable alerts for security analysts.

3. Traffic Data / Dataset Description

The prototype was trained and evaluated using the highly regarded CIC-IDS-2017 dataset, provided by the Canadian Institute for Cybersecurity. Unlike older, deprecated datasets (such as KDD99 or NSL-KDD), CIC-IDS-2017 contains completely realistic, modern network traffic profiles, including benign background traffic protocols (HTTP, HTTPS, FTP, SSH) and up-to-date attack scenarios.

For this assessment, the model focused on several specific traffic classes:

- **BENIGN:** Normal, routine network communications serving as our baseline.
- **PortScan:** Reconnaissance traffic designed to map open network ports.
- **DDoS (Distributed Denial of Service):** Volumetric attacks characterized by high packet rates intended to exhaust server resources.
- **FTP-Patator:** Brute-force attacks against file transfer protocols.
- **Infiltration:** Ambiguous, often covert traffic indicative of lateral movement or initial compromise.

4. System Architecture

The system architecture prioritizes speed, explainability, and privacy. It operates entirely on network flow statistics rather than payload content.

- **Ingestion Layer:** Raw .pcap files are converted into structured CSV formats containing bidirectional flow (Biflow) statistics.
- **Preprocessing Layer:** The system cleanses the data by handling missing values (NaNs), dropping infinite bounds (Infs), and standardizing feature scales. Categorical labels are transformed via a LabelEncoder.
- **Classification Layer:** A trained Random Forest model acts as the inference engine, scoring incoming flows against learned behavioral thresholds.
- **Analyst Interface:** The CLI processes the model's numerical output and generates human-readable alerts, complete with confidence scores and explainable reasoning.

5. Feature Extraction Methodology

Feature extraction is the most critical phase of anomaly detection. Because the system does not inspect payloads, it must rely exclusively on the mathematical shape of the conversation. The following key features were extracted and prioritized:

- **Flow Duration:** The total time a connection remains open. Extremely short durations often indicate rapid scanning (PortScan), while exceptionally long durations with low byte transfer suggest beaconing or slow-rate attacks (Infiltration).
- **Total Length of Fwd/Bwd Packets:** The total volume of data transmitted. Volumetric attacks (DDoS) exhibit massive spikes in these metrics compared to standard web browsing.
- **Fwd/Bwd Packet Length Mean/Max:** The average and maximum size of individual packets. Unexpected packet sizes can indicate payload delivery, exploitation attempts, or data exfiltration.
- **Flow Bytes/s and Packets/s:** The velocity of the network traffic. High rates strongly correlate with denial-of-service conditions or aggressive brute-force tools.

6. Detection Methodology

To analyze the extracted features, this prototype utilizes a Random Forest Classifier. A Random Forest is an ensemble learning method that constructs multiple decision trees during training

and outputs the mode of the classes for classification.

- **Why Random Forest?** Unlike deep neural networks, which act as "black boxes," Random Forest models provide high explainability through feature importance scoring. It allows security analysts to understand exactly *why* a flow was flagged. Additionally, Random Forests are highly resistant to overfitting and require less hyperparameter tuning than support vector machines (SVMs) or gradient boosting models.
- **Model Training:** The dataset was split using a standard 70/30 ratio (70% of the data used for training the model, 30% held back for testing and validation). The RandomForestClassifier was configured with `n_estimators=100` to balance performance and computational cost, and `max_depth=10` to prevent the model from memorizing training noise, forcing it to learn generalized behavioral rules.

7. Testing, Evaluation, and Results

The system was rigorously evaluated against the 30% holdout testing set. During this phase, the Random Forest model achieved exceptional aggregate metrics: an overall **Accuracy of 98.4%**, a **Precision of 97.9%**, and a **Recall of 98.1%**. These high-performance metrics were heavily driven by the model's ability to mathematically isolate volumetric attacks (like DDoS) with near-perfect precision, though recall dropped slightly on ambiguous classes like Infiltration.

To demonstrate the model's efficacy in a realistic operational environment, a CLI script (`run_scenarios.py`) was developed to simulate an analyst investigating live alerts.

Scenario 1 & 2: Baseline and Reconnaissance

The model successfully identified benign traffic with 100% confidence, establishing a stable baseline. It also correctly identified low-and-slow PortScan activity based on extended flow durations and irregular byte rates.

![CLI Output for Baseline and Recon](Screenshot 2026-08-28 122117.png)

Figure 1: CLI output demonstrating baseline traffic processing and successful identification of PortScan reconnaissance.

Scenario 3 & 4: DDoS and Brute Force

The system effectively detected DDoS traffic, correctly attributing the anomaly to abnormal mean packet lengths characteristic of the attack signature.

![CLI Output for DDoS and Brute Force](Screenshot 2026-08-28 122130.png)

Figure 2: CLI output demonstrating 100% confidence in volumetric DDoS anomaly detection, and handling of brute-force traffic.

8. Limitations

While the prototype achieved high accuracy, several limitations must be acknowledged:

- **Encrypted Traffic Analysis:** Because the system relies solely on flow statistics, it cannot identify attacks that perfectly mimic benign traffic patterns within an encrypted tunnel (e.g., an attacker manually downloading a file over an established SSH session).
- **Misclassification of Ambiguous Traffic:** The model occasionally misclassifies structurally similar attacks. For example, aggressive brute-force attacks (FTP-Patator) might be mislabeled as web-based exploitation (XSS) if their packet velocity and size profiles overlap.
- **Dataset Obsolescence:** Machine learning models are only as effective as their training data. As attack vectors evolve, the model will experience concept drift, requiring continuous retraining on newer datasets.

9. Future Improvements

To mature this prototype into a production-ready system, the following enhancements are proposed:

- **Hybrid Architecture Integration:** Combine this ML anomaly engine with a lightweight signature-based tool (like Snort or Suricata). The signature engine would handle known threats instantly, while the ML model acts as a fallback for zero-day behaviors.
- **Deep Learning Exploration:** Implement Long Short-Term Memory (LSTM) recurrent neural networks to better analyze the sequential, time-series nature of network flows, potentially improving the detection of advanced persistent threats (APTs) and infiltration.
- **SIEM Integration:** Export the CLI output to a standard logging format (like JSON or Syslog) to allow seamless ingestion into Security Information and Event Management (SIEM) platforms like Splunk or Elastic Security.

10. Project Deliverables

- **GitHub Repository:** <https://github.com/vinodhkumar232/megaminds-ids-assessment>
- **Video Demonstration:**
<https://drive.google.com/file/d/189ybtbg95KeodqyyvUSriqp9rFOeyXtJK/view?usp=sharing>